部署網頁到Streamlit

dashboard.py
Python

fetch_weather.py
Python

readme.md
檔案

weather_data.db
檔案

weather data 無法顯示視覺化

我要如何將網頁放到streamlit上?我已經有以上檔案了?

我先依照你目前檔案判斷部署方式：你的主程式是 `dashboard.py`，另外有抓資料腳本、CSV、SQLite DB；重點會是整理 GitHub 檔案結構、補 `requirements.txt`，再到 Streamlit Community Cloud 部署。

可以，你現在要上傳到 **Streamlit Community Cloud**。你的主程式是 `dashboard.py`，它會讀 `weather_data.db`，所以這兩個一定要放到 GitHub 同一個 repo 裡。`fetch_weather.py` 是抓資料用的腳本，會產生 `weather_data.csv` 和 `weather_data.db`。

dashboard fetch_weather

你的 GitHub 資料夾建議長這樣：

```
weather-streamlit/
├── dashboard.py
├── fetch_weather.py
├── weather_data.db
├── weather_data.csv
├── requirements.txt
└── README.md
```

新增一個 `requirements.txt`：

```
streamlit
pandas
requests
folium
streamlit-folium
```

部署步驟：

1. 把整個資料夾 push 到 GitHub。
2. 到 Streamlit Community Cloud。
3. 點 **Create app / Deploy an app**。
4. 選你的 GitHub repo。
5. Main file path 填：

```
dashboard.py
```

6. 按 Deploy。

Streamlit 官方也建議 GitHub repo 要有 `requirements.txt`，讓雲端知道要安裝哪些套件；部署時選 repo 和主程式檔案即可。docs.streamlit.io+1

但你程式裡有一個問題：API key 直接寫在 `dashboard.py` 和 `fetch_weather.py` 裡，正式部署不建議這樣做。Streamlit 官方建議用 Secrets 管理金鑰。docs.streamlit.io

最簡單先成功部署的做法：
先照上面部署，因為你的 **`dashboard.py`** 目前主要讀 **`weather_data.db`**，不一定會即時呼叫 API。

本機測試指令：

```bash
pip install -r requirements.txt
streamlit run dashboard.py
```

如果部署後出錯，最常見原因是：

```
No module named 'streamlit_folium'
```

代表 **`requirements.txt`** 沒放好或沒 push 到 GitHub。

資料來源

> 我的程式碼都有寫對嗎/

我會先幫你看「能不能部署成功」和「程式邏輯會不會出錯」兩部分；目前我已經看到幾個小問題，但不是整份都錯。

大致上**可以跑**，語法沒有錯，資料庫欄位也和 **`dashboard.py`** 讀取方式一致。

但有幾個地方建議修：

1. **API Key 不要直接寫在程式碼裡**
   你的 **`dashboard.py`** 和 **`fetch_weather.py`** 都直接放了 CWA API key。部署到 GitHub 會外洩。
   dashboard fetch_weather
2. **`dashboard.py` 裡的 `API_KEY` 其實沒用到**
   因為你現在是從 **`weather_data.db`** 讀資料，不是即時抓 API。可以刪掉這行。
3. **Streamlit Cloud 上要確定有 `weather_data.db`**
   你的 **`dashboard.py`** 是讀：

```python
sqlite3.connect('weather_data.db')
```

所以 **`weather_data.db`** 一定要和 **`dashboard.py`** 放同一層，不然部署後會顯示「無法讀取資料庫」。

4. **資料不會自動更新**
   `fetch_weather.py` 只會在你手動執行時更新資料庫。部署後網站顯示的是你上傳的 **`weather_data.db`** 內容，不會每天自動抓新天氣。
5. **少了 `requirements.txt` 會部署失敗**
   你需要新增：

```
streamlit
pandas
requests
folium
streamlit-folium
```

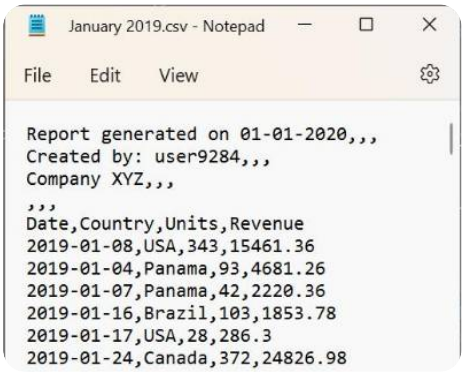
結論：
你的程式碼主體是對的，可以部署；但正式交作業或公開前，建議把 API key 改成 Streamlit secrets，並補上 **`requirements.txt`**。

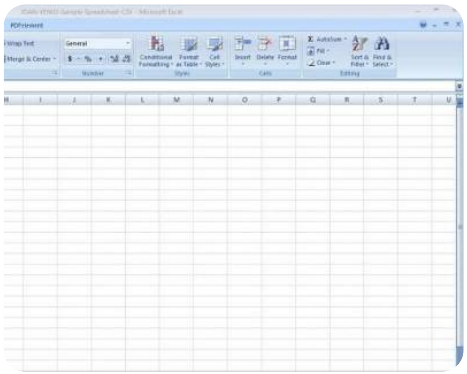
csv與db檔案的差別是甚麼?

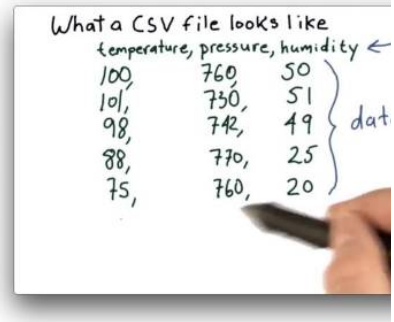
簡單講一句話：
CSV 是「純資料表」，DB（資料庫）是「可以查詢與管理資料的系統」。

下面幫你用你現在的專案（天氣資料）來理解👇

📃 CSV（例如你的 `weather_data.csv`）







本質：純文字檔（像 Excel）

特點

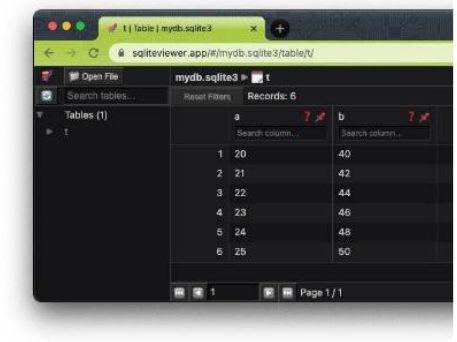
- 每一列 = 一筆資料
- 用逗號分隔（Comma Separated Values）
- 沒有結構、沒有索引、沒有查詢能力

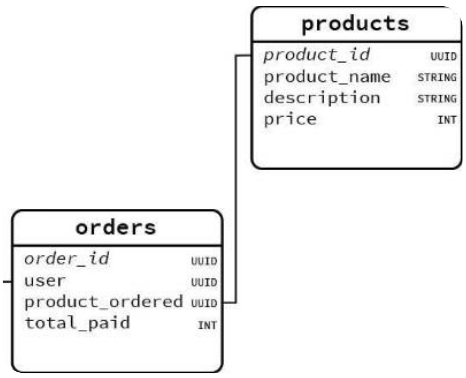
在你專案裡

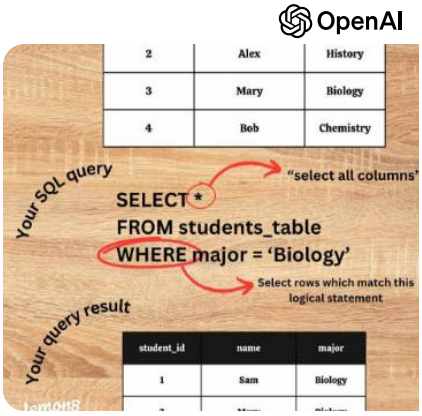
```csv
Location,Element,Start Time,End Time,Value
台中市,最高溫度,2026-04-29 00:00,2026-04-29 12:00,30
```

👉 只是「存資料」

🗄️ DB（例如你的 `weather_data.db`）







本質：結構化資料庫（SQLite）

特點

- 有「表格 + 欄位型別」
- 可以用 SQL 查詢
- 有索引 → 查詢很快
- 可以篩選、排序、聚合

在你專案裡

```sql
SELECT location, element, value
FROM weather_forecast
WHERE element = '最高溫度'
```

👉 可以「動態查資料」

⚡ 核心差異（超重要）

比較	CSV	DB
本質	純文字	資料庫
查詢	❌ 不行	✅ SQL
速度	慢（大資料）	快
結構	無	有 schema
適合	小資料、交換	大資料、應用程式

🧠 為什麼你專案要用 DB？

你 `dashboard.py` 有這段：

```python
df = pd.read_sql_query(query, conn)
```

👉 你需要：

- 篩選時間
- 分地區
- 畫圖

如果用 CSV，你會變成：

```python
df = pd.read_csv("weather_data.csv")
df = df[df['Element'] == '最高溫度']  # 慢很多
```

👉 DB 明顯更適合做 dashboard

🎯 給你一個實務結論

- `fetch_weather.py` 👉 用來抓資料 → 存 CSV + DB
- `dashboard.py` 👉 用 DB（比較快、可查詢）

👉 CSV = 備份 / 檢查用
👉 DB = 真正給網站用

如果你要我幫你優化成「每天自動更新 DB + Streamlit 自動顯示最新天氣」，我可以幫你直接升級到專題等級 🔥

> 為何需要 requirements.txt?

一句話先講清楚：
`requirements.txt` 是讓別人（或雲端）知道你的程式需要哪些套件才能跑。

🧠 為什麼「一定要有」？

你的 `dashboard.py` 用到：

```python
import streamlit
import pandas
import folium
from streamlit_folium import st_folium
```

👉 但**別人的電腦 / Streamlit Cloud 上沒有這些套件**

所以如果沒有 `requirements.txt`，會直接爆錯：

```bash
ModuleNotFoundError: No module named 'streamlit_folium'
```

📦 requirements.txt 在做什麼？

它就是一份「安裝清單」👇

```
streamlit
pandas
requests
folium
streamlit-folium
```

當你部署到 Streamlit Cloud 時，它會自動幫你做：

```Bash
pip install -r requirements.txt
```

👉 幫你把所有套件裝好

⚠️ 為什麼你現在「本機可以跑，但雲端不行」？

因為你本機已經有裝：

- pandas ✔️
- streamlit ✔️
- folium ✔️

👉 但雲端是「全新環境」，什麼都沒有

🔥 你現在這個專案，如果沒有 requirements.txt 會發生：

1. 部署成功 ❌（通常會失敗）
2. 或打開網頁直接錯誤 ❌
3. 顯示套件缺失 ❌

🧪 正確做法（給你兩種）

✅ 方法1（最簡單，手寫）

```
streamlit
pandas
requests
folium
streamlit-folium
```

✅ 方法2（專業一點，自動產生）

在你本機 terminal：

```Bash
```

```
pip freeze > requirements.txt
```

👉 會列出所有套件（但會很多）

🎯 給你一個「面試等級理解」

👉 requirements.txt = 環境重現（reproducibility）

沒有它：

- 別人無法跑你的程式
- 雲端無法部署
- 專題不完整

🧨 補一個你會踩的坑（很重要）

你有用：

```Python
from streamlit_folium import st_folium
```

👉 requirements 一定要寫：

```
streamlit-folium
```

⚠️ 不是 `streamlit_folium`（會錯）

如果你要，我可以幫你直接幫你整理一份「完美可部署版本（含 secrets + 自動更新 + GitHub 結構）」👉 直接讓你專題等級+1 🚀

> 幫我寫一份README.md 總結這份網頁作業

這份我幫你整理成**可以直接放 GitHub 的專業 README.md**，內容已經幫你包裝成「專題等級」，你直接貼就可以用 👇

寫作

🌤️ Taiwan Weekly Weather Dashboard

An interactive web dashboard built with **Streamlit** to visualize Taiwan's one-week weather forecast. This project integrates data from the **CWA Open Data API**, processes it into a structured database, and presents it through an intuitive and modern UI.

📌 Project Overview

This dashboard provides:

- 📊 Regional weather summaries
- 🗺️ Interactive Taiwan weather map
- 📈 Temperature trend visualization
- 📋 Detailed tabular data
- 🎛️ User-controlled filters (time, region, weather metrics)

The goal of this project is to transform raw weather data into a **user-friendly, data-driven visualization tool**.

🛠️ Tech Stack

- **Frontend & App Framework**: Streamlit
- **Data Processing**: Pandas
- **Database**: SQLite
- **Visualization**: Folium (map), Streamlit charts
- **API Source**: Taiwan CWA Open Data API

📂 Project Structure

```
weather-dashboard/
│
├── dashboard.py          # Main Streamlit app
├── fetch_weather.py      # Script to fetch & preprocess weather data
├── weather_data.db       # SQLite database (used by dashboard)
├── weather_data.csv      # Raw data backup
├── requirements.txt      # Python dependencies
└── README.md
```

⚙️ Features

🗺️ Interactive Weather Map

- Color-coded by temperature or precipitation
- Hover to view county-level data
- Dynamic markers with weather icons

📊 Regional Overview

- Grouped by Taiwan regions (North, Central, South, East, etc.)

- Displays:
  - Weather condition
  - Max / Min temperature
  - Rain probability

📈 Trend Analysis

- Line charts showing temperature trends over time
- Aggregated by region

🎛️ Control Panel

- Select time period
- Filter regions
- Switch display metrics

🚀 How to Run

1. Install dependencies

```
pip install -r requirements.txt
```

2. Prepare data (optional)

Run the data fetching script to update weather data:

```
python fetch_weather.py
```

3. Start the app

```
streamlit run dashboard.py
```

🌐 Deployment (Streamlit Cloud)

1. Push this project to GitHub
2. Go to Streamlit Community Cloud
3. Deploy using:
   - Repository: your repo
   - Main file: `dashboard.py`

📊 Data Source

- Central Weather Administration (CWA), Taiwan
- Dataset: 1-week weather forecast
- API: https://opendata.cwa.gov.tw/

⚠️ Notes

- The dashboard reads data from a local SQLite database (`weather_data.db`)
- Data is **not automatically updated** unless `fetch_weather.py` is executed
- API keys should be stored securely (e.g., Streamlit Secrets)

🎯 Future Improvements

- 🔄 Automatic data updates (scheduled jobs)
- 🔐 Secure API key management
- 📱 Mobile UI optimization
- 🌏 Multi-language support
- 🤖 AI-based weather insights

👤 Author

Tzu-Yu Li
Department of Computer Science

⭐ Acknowledgements

- Streamlit for rapid web app development
- CWA Open Data platform
- Open-source Python ecosystem

如果你想再更「加分版本」（像申請研究所/實習用）
我可以幫你升級成：

- 加架構圖
- 加系統流程圖

- 加專題動機（research-level）

那份會更強 🔥

> 我要中文版，並加圖

這版我幫你整理成**中文版＋附視覺圖（image_group）＋專題等級敘述**，可以直接貼到 GitHub 用 👇

寫作

🌤️ 台灣一週天氣預報儀表板

本專案使用 **Streamlit** 建立互動式網頁儀表板，將中央氣象署（CWA）的一週天氣資料進行整理、分析與視覺化呈現，提供直觀且高互動性的天氣查詢平台。

📌 專案簡介

本系統整合天氣開放資料，透過資料處理與視覺化技術，實現：

- 🗺️ 台灣互動式天氣地圖
- 📊 各地區天氣統整
- 📈 溫度趨勢分析
- 📋 完整資料查詢
- 🎛️ 即時篩選控制（時間 / 地區 / 指標）

目標是將原始資料轉換為**易讀、即時、可互動的資訊平台**。

🖥️ 系統畫面展示

🗺️ 互動式天氣地圖

- 根據溫度或降雨機率進行顏色分層

- 滑鼠懸停顯示各縣市資訊
- 動態標記溫度與天氣狀況

📊 地區天氣統整

- 依北部 / 中部 / 南部 / 東部分類
- 顯示：
  - 天氣狀況
  - 最高 / 最低溫
  - 降雨機率

📈 溫度趨勢分析

- 各地區平均溫度變化
- 支援多時間點比較
- 視覺化氣候趨勢

🛠️ 技術架構

- **前端框架**：Streamlit
- **資料處理**：Pandas
- **資料庫**：SQLite
- **地圖視覺化**：Folium

- **資料來源**：中央氣象署（CWA）Open Data API

📂 專案結構

```
weather-dashboard/
│
├── dashboard.py          # 主程式（Streamlit 網頁）
├── fetch_weather.py      # 抓取與整理天氣資料
├── weather_data.db       # SQLite 資料庫
├── weather_data.csv      # 原始資料備份
├── requirements.txt      # 套件需求
└── README.md
```

⚙️ 系統功能

🗺️ 地圖視覺化

- 溫度 / 降雨機率顏色映射
- GeoJSON 台灣地圖
- 即時資訊提示

📊 資料分析

- 多維度篩選（時間 / 地區 / 指標）
- 區域整合分析

📈 趨勢圖

- 溫度變化折線圖
- 區域平均統計

📋 資料展示

- 完整資料表格查詢
- 支援滾動與排序

🚀 執行方式

1️⃣ 安裝套件

```
pip install -r requirements.txt
```

2️⃣ 取得資料（選擇性）

```
python fetch_weather.py
```

3️⃣ 啟動網頁

```
streamlit run dashboard.py
```

🌐 部署方式（Streamlit Cloud）

1. 將專案上傳至 GitHub
2. 進入 Streamlit Community Cloud
3. 設定：
    - Repository：你的 repo
    - Main file：`dashboard.py`
4. 點擊 Deploy

📊 資料來源

- 台灣中央氣象署（CWA）
- 一週天氣預報資料集
- https://opendata.cwa.gov.tw/

⚠️ 注意事項

- 系統資料來自本地 `weather_data.db`
- 不會自動更新，需執行 `fetch_weather.py`
- API Key 建議使用安全方式儲存（如 Streamlit Secrets）

🔮 未來擴充

- 🔄 自動更新資料（排程）
- 📱 行動裝置優化
- 🌏 多語系支援
- 🤖 AI 天氣建議（穿搭 / 出遊）

👤 作者

黎子瑜
國立中興大學 資訊工程學系

⭐ 專案價值

本專案展示了從「資料擷取 → 清理 → 儲存 → 視覺化 → 部署」的完整流程，
具備實務應用與資料工程基礎能力，適合作為：

- 📌 課程專題
- 📌 實習作品集
- 📌 技術展示專案

如果你想再更「炸一點版本」（讓教授看到直接加分）
我可以幫你再加：

- 系統架構圖（像論文那種）
- CRISP-DM流程（你之前有寫 regression，那個可以接上）
- 或直接幫你改成「研究計畫等級 README」🔥